第36章源码阅读

由正式源文件生成的静态阅读副本。返回正文 · 运行说明

---

src/main/java/org/example/jvminternals/ch36/CaseWorkload.java

```
package org.example.jvminternals.ch36;

import java.lang.foreign.Arena;
import java.lang.management.ManagementFactory;
import java.time.Instant;
import java.util.Arrays;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.Executors;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Future;
import java.util.concurrent.Semaphore;
import java.util.concurrent.TimeUnit;

/** 明确授权的有限对照；模式固定，不接受无上限任务数或分配参数。 */
public final class CaseWorkload {
    private static final Object GATE = new Object();
    private static volatile byte[] escaped;
    private static int completed;

    private CaseWorkload() {
    }

    /** 独立进程隔离模式，启动 / 预热另复用第31章固定应用。 */
    public static void main(String[] args) throws Exception {
        System.out.printf("案例 pid=%d mode=%s time=%s processors=%d%n",
                ProcessHandle.current().pid(), args[0], Instant.now(),
Runtime.getRuntime().availableProcessors());
        switch (args[0]) {
            case "allocate", "reuse" -> allocation(args[0].equals("reuse"));
            case "locked", "narrow", "platform-budget", "virtual-budget" ->
concurrency(args[0]);
            case "retained", "scoped" -> nativeMemory(args[0].equals("scoped"));
            default -> throw new IllegalArgumentException("未知案例模式");
        }
    }

    private static void allocation(boolean reuse) {
```

```
        byte[] scratch = new byte[4096];
        // 两种模式相同热身操作数；不把热身误记为测量区间。
        for (int index = 0; index < 10_000; index++) {
            touch(reuse ? scratch : new byte[4096], index);
        }
        var bean = (com.sun.management.ThreadMXBean) ManagementFactory.getThreadMXBean();
        long threadId = Thread.currentThread().threadId();
        long allocatedBefore = bean.getThreadAllocatedBytes(threadId);
        long[] latencies = new long[40_000];
        long start = System.nanoTime();
        for (int index = 0; index < latencies.length; index++) {
            long operationStart = System.nanoTime();
            touch(reuse ? scratch : new byte[4096], index);
            latencies[index] = System.nanoTime() - operationStart;
        }
        long elapsed = System.nanoTime() - start;
        long allocated = bean.getThreadAllocatedBytes(threadId) - allocatedBefore;
        report(latencies, elapsed, "threadAllocatedBytes=" + allocated);
    }

    private static void touch(byte[] bytes, int index) {
        bytes[index & 4095] = (byte) index;
        // 让数组逃逸，限定本例观察的是实际分配，非标量替换后的逻辑对象。
        escaped = bytes;
    }

    private static void concurrency(String mode) throws Exception {
        boolean platform = mode.equals("platform-budget");
        boolean budgeted = mode.endsWith("budget");
        Semaphore budget = new Semaphore(4);
        long[] latencies = new long[64];
        long start = System.nanoTime();
        try (ExecutorService executor = platform ? Executors.newFixedThreadPool(4)
                : Executors.newVirtualThreadPerTaskExecutor()) {
            List<Future<?>> futures = new ArrayList<>();
            for (int index = 0; index < latencies.length; index++) {
                int slot = index;
                long submitted = System.nanoTime();
                futures.add(executor.submit(() -> {
                    try {
                        task(mode, budgeted, budget);
                        latencies[slot] = System.nanoTime() - submitted;
                    } catch (InterruptedException interrupted) {
                        Thread.currentThread().interrupt();
                        throw new IllegalStateException("案例任务被中断", interrupted);
                    }
                }));
            }
            for (Future<?> future : futures) {
                future.get();
            }
```

```
        }
        report(latencies, System.nanoTime() - start, "completed=" + completed);
    }

    private static void task(String mode, boolean budgeted, Semaphore budget) throws
InterruptedException {
        if (budgeted) {
            budget.acquire();
        }
        try {
            if (mode.equals("locked")) {
                synchronized (GATE) {
                    // 有意把等待放入临界区，作为缩窄持锁范围的前态。
                    TimeUnit.MILLISECONDS.sleep(10);
                    completed++;
                }
            } else {
                TimeUnit.MILLISECONDS.sleep(10);
                synchronized (GATE) {
                    completed++;
                }
            }
        } finally {
            if (budgeted) {
                budget.release();
            }
        }
    }

    private static void nativeMemory(boolean scoped) throws InterruptedException {
        try (Arena retained = Arena.ofConfined()) {
            for (int index = 0; index < 32; index++) {
                if (scoped) {
                    try (Arena oneTask = Arena.ofConfined()) {
                        oneTask.allocate(256 * 1024).fill((byte) index);
                    }
                } else {
                    retained.allocate(256 * 1024).fill((byte) index);
                }
            }
            System.out.println("资源快照窗口 requestedBytes=8388608 scoped=" + scoped);
            Thread.sleep(5_000);
        }
        System.out.println("所有Arena关闭");
        Thread.sleep(3_000);
    }

    private static void report(long[] values, long elapsed, String detail) {
        Arrays.sort(values);
        System.out.printf("测量 count=%d wallNs=%d p50Ns=%d p99Ns=%d maxNs=%d %s%n",
                values.length, elapsed, values[values.length / 2],
```

```
                values[(int) Math.ceil(values.length * 0.99) - 1], values[values.length -
1], detail);
    }
}
```

---

benchmarks/src/main/java/org/example/jvminternals/ch36/WorkBenchmark.java

```
package org.example.jvminternals.ch36;

import java.util.concurrent.TimeUnit;
import org.openjdk.jmh.annotations.Benchmark;
import org.openjdk.jmh.annotations.BenchmarkMode;
import org.openjdk.jmh.annotations.Fork;
import org.openjdk.jmh.annotations.Measurement;
import org.openjdk.jmh.annotations.Mode;
import org.openjdk.jmh.annotations.OutputTimeUnit;
import org.openjdk.jmh.annotations.Scope;
import org.openjdk.jmh.annotations.State;
import org.openjdk.jmh.annotations.Warmup;

/** 明确授权的小规模微基准，返回结果防止整个计算被作为死代码删除。 */
@State(Scope.Thread)
@BenchmarkMode(Mode.AverageTime)
@OutputTimeUnit(TimeUnit.NANOSECONDS)
@Warmup(iterations = 2, time = 200, timeUnit = TimeUnit.MILLISECONDS)
@Measurement(iterations = 3, time = 200, timeUnit = TimeUnit.MILLISECONDS)
@Fork(value = 2, jvmArgsAppend = {"-Xms64m", "-Xmx64m"})
public class WorkBenchmark {
    private long input = 7;

    /** 可变线程私有状态限制常量折叠；不在计时路径逐次写日志。 */
    @Benchmark
    public long compute() {
        input = input * 1_664_525 + 1_013_904_223;
        return input;
    }
}
```

../ch31-aot-cache/src/main/java/org/example/jvminternals/ch31/CacheJourney.java

```java
package org.example.jvminternals.ch31;

import java.lang.management.ManagementFactory;

/** 有限请求流让训练画像与部署新路径可见，不作性能基准。 */
public final class CacheJourney {
    private interface Operation {
        long apply(long value);
    }

    private static final class Add implements Operation {
        @Override
        public long apply(long value) {
            return value + 7;
        }
    }

    private static final class Multiply implements Operation {
        @Override
        public long apply(long value) {
            return value * 3;
        }
    }

    private CacheJourney() {
    }

    /** 输入只选择两条正常业务路径；循环上限固定，不计算吞吐或稳定期。 */
    public static void main(String[] args) {
        String mode = args.length == 0 ? "add" : args[0];
        mark("入口 mode=" + mode);
        Operation operation = "multiply".equals(mode) ? new Multiply() : new Add();
        mark("就绪");
        long result = 0;
        for (int request = 0; request < 60_000; request++) {
            result += route(operation, request);
        }
        System.out.printf("CH31 请求流结束 result=%d%n", result);
        mark("结束");
    }

    private static long route(Operation operation, long value) {
        return operation.apply(value);
```

```
    }

    private static void mark(String phase) {
        System.out.printf("CH31 %s uptimeMs=%d%n", phase,
                ManagementFactory.getRuntimeMXBean().getUptime());
    }
}
```